

BluBLAS: Hand-Tuned GEMM Kernels for Ada Lovelace Tensor Cores Across FP32, FP16, and BF16

A Study of Precision Emulation, Bank-Conflict-Free Swizzling, and L2-Aware Scheduling for GPT-scale Training Workloads

BluBridge Team
contact@blubridge.ai

Abstract

General matrix multiplication (GEMM) dominates the runtime of transformer training and inference, yet vendor libraries such as cuBLAS are tuned for an “average” workload and leave measurable performance on the table for the specific tile shapes that arise inside GPT-scale models. We present **BluBLAS**, a custom CUDA GEMM library written in explicit PTX and CUDA C++ for the NVIDIA Ada Lovelace (SM89) architecture, with an Ampere (SM86) fallback. The library covers three precisions FP32 (via TF32 emulation), FP16, and BF16 each in all three operand layouts (NN, NT, TN) required by the forward and backward passes of a linear layer. We make three contributions. First, we describe a single-instruction operand-bias trick (+0x1000) that recovers near round-to-nearest accuracy from the hardware’s truncating FP32→TF32 conversion, combined with a fused-multiply-add epilogue that matches cuBLAS’s single-rounding guarantee. Second, we give layout-dependent XOR swizzling formulas for shared memory, paired with `ldmatrix` hardware gather and a two-phase swizzled epilogue, that eliminate shared-memory bank conflicts on all supported layouts. Third, we combine a six-stage `cp.async` software pipeline with column-strip CTA grid swizzling that keeps the operand matrix resident in the A6000 Ada’s 96 MB L2 cache, and a Split-K decomposition that restores occupancy on the low-parallelism gradient shapes. On an A6000 Ada we reach 217 TFLOPS (FP16) and 226 TFLOPS (BF16) on transformer gradient layers, corresponding to speedups of up to +26% and +40% over cuBLAS, while matching its numerical accuracy. We also report and analyze the shapes where cuBLAS still wins, and outline the path to closing that gap.

1 Introduction

The compute budget of large language model training is overwhelmingly spent in dense matrix multiplication. Within a single transformer block, the QKV projection, attention output projection, and the two MLP linear layers are all GEMMs, and each appears three times across

a training step: once in the forward pass and twice in the backward pass (one product for the input gradient and one for the weight gradient). These three uses correspond to three distinct operand layouts commonly denoted NN, NT, and TN that stress the memory system in different ways.

In practice, almost all of this multiplication is dispatched to NVIDIA’s cuBLAS library. cuBLAS is an excellent general-purpose product, but it is a black box tuned to perform well *on average* across an enormous space of problem shapes and hardware generations. For the narrow but high-value set of shapes that recur inside a fixed model architecture and on a single, known GPU a hand-tuned kernel can exploit architecture-specific features that a general library cannot afford to specialize for. The question this work answers is concrete: *on an A6000 Ada (RTX 6000 Ada Generation, Ada Lovelace, SM89), running GPT-2 scale layer shapes, how much performance is recoverable by writing the kernels by hand, and which optimizations are responsible?*

We answer it with **BluBLAS**, a GEMM library in which every kernel is written in explicit PTX and CUDA C++ rather than relying on cuBLAS or the WMMA C++ API. This level of control is what makes the optimizations below possible: the WMMA API does not expose the exact `m16n8k8` TF32 MMA shape we need, does not expose `ldmatrix`, and does not give direct control over the `cp.async` pipeline groups.

Contributions. This paper makes the following contributions:

1. **Accurate TF32 emulation (§4).** Ada Tensor Cores do not accumulate native FP32. We show how a single integer add of 0x1000 to each operand’s bit pattern, applied before the hardware’s truncating TF32 conversion, recovers near round-to-nearest behavior, and how a fused-multiply-add epilogue preserves cuBLAS’s single-rounding accuracy.
2. **Bank-conflict-free half-precision pipelines (§5).** We derive layout-specific XOR swizzles for shared memory that, together with `ldmatrix` gather and a two-phase swizzled epilogue, remove all shared-

memory bank conflicts on NN, NT, and TN layouts.

3. **Memory-system co-design (§6–§7).** We combine a six-stage `cp.async` pipeline, column-strip CTA grid swizzling for L2 residency, register double-buffering, and a Split-K decomposition that restores SM occupancy on low-parallelism gradient shapes.
4. **An honest evaluation (§9).** On real GPT-2 layer shapes we report up to +40% over cuBLAS in BF16, and we identify and explain the small forward shapes where cuBLAS remains faster.

2 Background

2.1 The GEMM Operation and Its Layouts

All kernels implement the standard BLAS contract $C = \alpha \text{op}(A) \text{op}(B) + \beta C$, where $\text{op}(\cdot)$ is identity (N) or transpose (T). For a linear layer $Y = XW^\top$ the three passes give rise to three layouts:

- **NN** (forward): $Y = XW^\top$, the activation product.
- **NT** (input gradient): $dX = dYW$.
- **TN** (weight gradient): $dW = dY^\top X$.

Each layout changes which operand is contiguous along the contraction dimension K , and therefore changes the shared-memory access pattern and the swizzle required to keep it conflict-free.

2.2 Ada Lovelace Tensor Cores and the MMA Instruction

Tensor Cores execute a warp-synchronous matrix-multiply-accumulate (MMA) on a small tile per instruction. The relevant PTX forms are `mma.sync.aligned.m16n8k8.row.col.f32.tf32.tf32.f32` for TF32 and `m16n8k16` for FP16/BF16. Each instruction consumes register fragments distributed across the 32 lanes of a warp and accumulates into FP32 registers. A 128×128 CTA tile with four warps issues $(128/16) \times (128/8) = 128$ TF32 MMA calls per K -tile.

Crucially, Ada Tensor Cores have *no* native FP32 datapath; FP32 GEMM on the CUDA cores peaks near 35 TFLOPS on this class of device, far below Tensor Core throughput. TF32 an 8-bit exponent (same range as FP32) with a 10-bit mantissa is the widest format the Tensor Cores accelerate, which motivates the emulation strategy of §4.

2.3 Fragment–Lane Mapping and Tile Decomposition

The MMA is warp-synchronous: the input and output matrices are not held by a single thread but are distributed across the 32 lanes of the warp in a fixed, hardware-defined pattern. For the `m16n8k16` accumulator, each

lane owns four FP32 values arranged as two 2×1 column pairs, with lane ℓ holding rows $\{\ell/4, \ell/4+8\}$ and column $2(\ell \bmod 4)$ of the 16×8 output tile. Understanding this mapping is what makes the swizzled epilogue (§5) necessary: the lane-major register order does not correspond to row-major global order, so a direct store scatters.

A kernel’s work is decomposed hierarchically: the CTA owns a $B_M \times B_N$ output tile; the warps partition that tile into per-warp sub-tiles of $WT_M \times WT_N$; and each warp issues $MMA_M \times MMA_N = (WT_M/16) \times (WT_N/8)$ MMA calls per K -slice. For the 256×128 , four-warp configuration with a 2×2 warp grid, each warp covers a 128×64 sub-tile and issues $8 \times 8 = 64$ MMA calls per K -slice of 16. The arithmetic intensity of the CTA FLOPs per byte loaded from global memory rises with $B_M B_N / (B_M + B_N)$, which is why the larger 256×128 tile is preferred whenever the problem is large enough to fill the SMs: it amortizes each loaded element over more MMA work.

2.4 Shared Memory Banks and `ldmatrix`

Shared memory is organized into 32 banks of 4 bytes, giving a 128-byte bank period. When a warp’s threads load 16-byte vectors that all fall in the same 4-bank group, the accesses serialize into a multi-way bank conflict. The `ldmatrix` instruction performs a warp-cooperative, optionally transposing, load of four 8×8 `b16` tiles exactly the fragment layout the MMA expects but it does not by itself avoid conflicts; the *address mapping* of the shared buffer must be chosen to do so. This is the role of XOR swizzling (§5).

2.5 Asynchronous Copy

The Ampere/Ada `cp.async` instruction copies directly from L2/DRAM into shared memory without staging through registers, and its completion can be tracked in groups. This is the building block of the software pipeline in §6 and is not exposed with full control through the high-level CUDA APIs.

3 Library Overview

BluBLAS exposes a cuBLAS-like C API behind a lightweight context handle that caches the device’s SM version and SM count, queried once at handle creation. Every launch dispatches in two stages: (i) an architecture dispatch that routes to SM89 or SM86 kernels, and (ii) a shape-based heuristic that selects a tile configuration and decides whether to apply Split-K (§7). The kernels themselves are generated from a small set of templates one per precision parameterized by tile dimensions (B_M, B_N, B_K), pipeline depth (STAGES), and thread count, so that the NN/NT/TN variants of a precision share almost all their code and differ only in load indexing and swizzle formula.

Table 1: Supported operations by precision. “All” = NN, NT, and TN.

Precision	Fwd	Bwd	AddMM	GEMV	Split-K
FP32 (TF32)	All	All	✓	✓	$\leq 16\times$
FP16	All	All	✓	✓	$\leq 16\times$
BF16	All	All	✓	✓	$\leq 16\times$

Three architectural pillars, developed in the next three sections, account for the bulk of the measured speedups:

1. FP32/TF32 precision emulation with a single-add rounding correction.
2. Zero-conflict half-precision pipelines via layout-dependent swizzling.
3. L2-aware CTA scheduling combined with a deep `cp.async` pipeline.

Table 1 summarizes the operations supported per precision. All three full precisions implement the NN/NT/TN layouts, AddMM, GEMV, strided-batched execution, and Split-K up to $16\times$; FP64 is provided only as a basic strided-batched path.

4 FP32 Kernel: TF32 Emulation

4.1 Why Emulation Is Necessary

Because the Tensor Cores cannot accumulate FP32 directly, a naive FP32 GEMM is confined to the CUDA cores and roughly 35 TFLOPS. The alternative is to feed the operands through the TF32 MMA datapath, which runs at FP16/BF16-class throughput. The cost is precision: converting an FP32 value to TF32 discards the lower 13 of its 23 mantissa bits.

4.2 The Operand-Bias Rounding Correction

The hardware FP32→TF32 conversion *truncates*: it simply drops mantissa bits [12:0]. Truncation is a biased rounding mode it always rounds toward zero in magnitude so over a long dot product ($K = 768$ or $K = 3072$) the error accumulates into a systematic deviation from the FP32 reference, rather than averaging out.

We correct this with a single integer addition before truncation. Let x be the 32-bit pattern of an FP32 operand. The weight of TF32’s least-significant retained bit (bit 13) is 2^{13} ULP_{FP32}, so half of one TF32 ULP is $2^{12} = 0x1000$. Adding this constant to the integer bit pattern and then truncating implements round-to-nearest:

$$\hat{x}_{\text{TF32}} = \text{trunc}_{13}(x + 0x1000). \quad (1)$$

The behavior splits into three cases by the value of the discarded field $x_{[12:0]}$:

- If $x_{[12:0]} > 2^{12}$ (bit 12 set with residue), the add carries into bit 13: the mantissa rounds *up*.
- If $x_{[12:0]} < 2^{12}$, no carry occurs: equivalent to rounding *down*.
- If $x_{[12:0]} = 2^{12}$ exactly (a tie), the add carries, giving round-half-up.

This is exact round-to-nearest for the non-tie cases and round-half-up for ties; it differs from IEEE-754 round-to-nearest-*even* only in tie handling, which is statistically negligible over a GEMM and costs a single `IADD`. Because each operand is truncated independently by the hardware, the bias is applied to *both* the A and B fragments:

```
reg[0] += 0x1000u; reg[1] += 0x1000u; // A fragment
reg[2] += 0x1000u; reg[3] += 0x1000u;
// ... and likewise for the B fragment
```

A Worked Example

Consider an operand whose FP32 bit pattern has retained TF32 mantissa ... 1010 followed by a discarded field $x_{[12:0]} = 0x1A00 (> 2^{12})$. Pure truncation drops the field and yields a value short of the true magnitude by $0x1A00$ ULP. Adding $0x1000$ first gives $x_{[12:0]} = 0x1A00 + 0x1000 = 0x2A00$, whose carry out of bit 13 increments the retained mantissa to ... 1011 before truncation the correctly rounded result. Repeated over a $K = 3072$ dot product, replacing a systematically-low truncation error with a zero-mean rounding error removes the accumulated drift that otherwise separates emulated TF32 from the FP32 reference; this is what lets the emulated path match cuBLAS within its reported tolerance while running on the Tensor Cores.

4.3 FMA-Fused Epilogue for Accuracy Parity

The epilogue computes $\alpha \text{acc} + \beta C_{\text{old}}$. Doing this as two separate operations rounds twice. We instead fuse both into a single correctly-rounded FMA, matching cuBLAS’s single-rounding guarantee, and store through vectorized `float2` (8-byte) transactions:

```
float f0 = __fmaf_rn(alpha, v0, 0.f);
float f1 = __fmaf_rn(alpha, v1, 0.f);
if (beta != 0.f) {
    float2 o = *(float2*)p;
    f0 = __fmaf_rn(beta, o.x, f0); // single rounding
    f1 = __fmaf_rn(beta, o.y, f1);
}
*(float2*)p = make_float2(f0, f1);
```

4.4 Layout-Dependent Swizzling (FP32)

A tile stored row-major as `As[r*BK+c]` collides catastrophically under the MMA access pattern. The MMA reads a *column strip* a fixed column `c` across many rows and for any stride that is a multiple of the 128-byte bank period, every row maps `c` to the same four banks, serializing the warp into an 8-way conflict. We resolve this with

Table 2: Decoding the swizzle of Eq. 2.

Field	Role	Typical value
PERIOD	rows before the pattern repeats	7 (period-8) or 3
SHIFT	$\log_2$ of the vector group size	2 (float4), 3 (8-wide)
CLAMP	row width - 1	BK-1, BN-1, BM-1

Table 3: FP32 store/read swizzle by layout. The read mask is the *same* formula re-applied (self-inverse); for NT/NN it is precomputed per MMA row, for TN it is inlined because it depends on the running k .

Layout	Matrix	Store formula	Shift / clamp
NT/NN	A	$c^{\wedge}(((r \& 7) < 2) \& (BK-1))$	< 2 , BK-1
TN	A	$c^{\wedge}((r \& 7) < 3)$	< 3 , none
NT	B	$c^{\wedge}(((r \& 7) < 2) \& (BK-1))$	< 2 , BK-1
TN/NN	B	$c^{\wedge}(((r \& 7) < 2) \& (BN-1))$	< 2 , BN-1

an XOR perturbation of the column index that is a pure function of the row:

$$sc = c \oplus ((r \& \text{PERIOD}) \ll \text{SHIFT}) \& \text{CLAMP}. \quad (2)$$

XOR is chosen over padding or load-time transposition for four reasons [4]: it is its own inverse, so the read path recovers logical positions by re-applying the same mask (§4.5); it never overflows, only flipping bits; it compiles to a single instruction folded into address arithmetic, costing no extra shared memory (unlike padding) and no extra buffer (unlike transposition); and its period can be tuned to match the conflict period exactly. Table 2 gives the meaning of each field of Eq. 2, and Table 3 instantiates it per layout.

Worked example (NT/NN A, $B_K=32$). A row holds eight float4 words spanning all 32 banks in 8 groups of 4, so the conflict period is 8 rows and the mask cycles $(r \& 7) < 2 \in \{0, 4, \dots, 28\}$. Reading the MMA column $c=0$ across rows 0–7 then lands at $sc = \{0, 4, 8, 12, 16, 20, 24, 28\}$, i.e. banks $\{0, 4, 8, 12, 16, 20, 24, 28\}$ all distinct, zero conflicts, versus an 8-way conflict without the swizzle. The TN layout transposes A to stride $B_M=128$, where the wider row requires an 8-element step (< 3) so the masks $\{0, 8, \dots, 56\}$ remain collision-free. The two FP32 mask constants follow directly: SMEM_MASK is $7 < 3$ for $B_K=32$ and $(BM \geq 64 ? 3 : 1) < 2$ for the clamped $B_K=16$ tiles.

4.5 The Read Path Is the Same Mask

The store and read swizzles are never derived separately. XOR is an involution: $(c \oplus \text{mask}) \oplus \text{mask} = c$. Writing the permuted address $\text{physical} = \text{logical} \oplus \text{mask}$ and later reading with the identical mask recovers the logical position exactly, so a single formula serves both directions. Two engineering choices follow. For the NT/NN layouts

Table 4: Production FP32 (TF32) tile configurations.

Cfg	B_M	B_N	B_K	Stages	Thr.	Use
L	256	128	32	2	256	large ($\geq 256 \times 128$)
M	128	128	16	6	128	medium ($\leq 1024^2$)
S	128	64	16	6	128	narrow N ($N \leq 64$)

the mask depends only on the warp’s row, so it is *precomputed* once into a small per-MMA-row array (`maskA[i]`) and hoisted out of the hot K -loop; for the TN layout the mask depends on the running contraction index k , so it is *inlined* and recomputed each step. This involution property is also why the two-phase epilogue (§5) can read back exactly what it scattered.

The production FP32 tiles are 256×128 (large), 128×128 (medium), and 128×64 (narrow- N), summarized in Table 4. The largest tile is launched with `__launch_bounds__(THREADS, 1)` so the compiler optimizes for a single CTA per SM and does not spill the double-buffered fragment registers to satisfy a higher occupancy target it cannot reach anyway.

5 Half-Precision Kernels (FP16 and BF16)

FP16 and BF16 share the `m16n8k16` MMA shape and differ only in operand type. Both accumulate in FP32 registers throughout the K -loop and narrow to 16-bit only at the epilogue half-precision accumulators would lose dynamic range or precision over $K = 3072$ dot products. We therefore describe them together and note the differences in Table 5.

5.1 ldmatrix Gather

Fragments are loaded with `ldmatrix.x4`, which reads four 8×8 `b16` tiles 16 bytes per lane, perfectly matched to the 16-byte `cp.async` granularity. The `.trans` variant transposes during the load itself, eliminating a software transpose for the TN layout:

```
// NN / NT: direct load
ldmatrix.sync.aligned.m8n8.x4.shared.b16
{ %0, %1, %2, %3 }, [ %4 ];
// TN: transposing load
ldmatrix.sync.aligned.m8n8.x4.trans.shared.b16
{ %0, %1, %2, %3 }, [ %4 ];
```

5.2 XOR Swizzling for Bank-Conflict Removal

The half-precision swizzle is the same Eq. 2 as FP32 but with two changes forced by the 2-byte element. First, the vector group is now 8 elements ($8 \times \text{bf16} = 16$ bytes), so the shift becomes < 3 rather than < 2 . Second and less obvious the conflict *period halves* for the same B_K : a 16-byte vector spans 4 banks, but a $B_K=32$ row of 16-bit

elements occupies only $32/2 = 16$ banks, i.e. $16/4 = 4$ vec-groups, so the NT/NN period is 4 rows ($\mathbf{r\&3}$), while the TN layout’s stride-128 row spans 64 banks $\rightarrow$ 8 groups $\rightarrow$ period 8 ($\mathbf{r\&7}$):

```
// NT/NN, stride BK=32 elems: period 4
int sc = c ^ ((r & 3) << 3); // masks {0,8,16,24}
// TN, stride 128 elems: period 8
int sc = c ^ ((r & 7) << 3); // masks {0,8,...,56}
```

Worked example (NT/NN A , $B_K=32$). The masks ($\mathbf{r\&3} < 3 = \{0, 8, 16, 24\}$) place the vec-8 loads of rows 0–3 into four *disjoint* 4-bank groups; reading MMA column $\mathbf{c=0}$ down those rows therefore issues a single transaction instead of a 4-way conflict. The TN case is analogous with eight masks $\{0, 8, \dots, 56\}$ over its wider row.

The read path is the same mask. Unlike FP32, the half-precision read is a hardware `ldmatrix`, but the conflict-avoidance still relies on each lane supplying a swizzled shared address. Because XOR is self-inverse (§4.5), the de-swizzle re-applies the identical formula at the logical coordinates the lane is reading e.g. TN- A reads $\mathbf{As[k*BM + (m^((k\&7)<<3))]}$ through the `.trans` variant. BF16 and FP16 share these masks because they share 16-bit packing density.

5.3 Two-Phase Swizzled Epilogue

After the K -loop each thread holds its MMA output fragments in FP32 registers, but the MMA register-to-element mapping does not match row-major global order, so a direct store would be uncoalesced. We stage through shared memory in two phases, reusing the same buffer that held the A/B tiles (safe because a `__syncthreads()` guarantees the last tile has been consumed).

Phase 1 writes each fragment to shared memory under a row swizzle $\mathbf{r \oplus ((c/8) \& 7)}$, scaling by α on the way:

```
int sr = r ^ ((c/8) & 7);
*(__half2*)&s_out[sr*BN + c] =
{__float2half(acc[i][j][0]*a),
 __float2half(acc[i][j][1]*a)};
*(__half2*)&s_out[(sr^8)*BN + c] =
{__float2half(acc[i][j][2]*a),
 __float2half(acc[i][j][3]*a)};
```

The $\mathbf{sr \oplus 8}$ term addresses the second 8-row group of the 16-row MMA tile: since the mask uses only bits [2:0] via $\mathbf{\&7}$, bit 3 is free, and XOR-ing it with 8 is equivalent to adding 8 while preserving the swizzle. The rotation selector is the *column group* $\mathbf{(c/8)\&7}$: for $B_N=128$ the staging row spans 64 banks $\rightarrow$ 8 column groups, so period-8 rotation sends each column group of a given logical row to a distinct physical row (e.g. logical row 0 lands at physical rows 0, 1, 2, ... for column groups 0, 1, 2, ...; row 1 at 1, 0, 3, ...), guaranteeing that threads in different column groups never share a bank. *Phase 2* reads the staged data back as 16-byte `int4` vectors and performs fully coalesced global stores (8 contiguous FP16 per transaction); the same $\mathbf{sr}$ formula un-swizzles on the way out. BF16 is

Table 5: FP16 vs. BF16 implementation differences. The templates are otherwise parallel.

Feature	FP16	BF16
MMA	<code>m16n8k16.f16</code>	<code>m16n8k16.bf16</code>
Accumulator	FP32	FP32
Vector type	<code>__half2</code>	<code>__nv_bfloat162</code>
Swizzle (NT/TN)	4 / 8 rows	4 / 8 rows
<code>ldmatrix</code>	<code>.b16</code>	<code>.b16</code>

Table 6: Production FP16/BF16 tile configurations.

Cfg	B_M	B_N	B_K	Stages	Thr.	Use
Large	256	128	32	3	256	QKV / FC fwd
Medium	128	128	32	5	128	attn. output
Small	64	64	32	6	128	small shapes
Narrow	128	64	32	6	128	rectangular

structurally identical with `__nv_bfloat162`; for the Split-K path the store becomes a vectorized `atomicAdd` on `__nv_bfloat162`.

The half-precision tiles span 256×128 (QKV/FC forward), 128×128 (attention output), 64×64 (small shapes), and a 128×64 rectangular tile, with deeper pipelines (5–6 stages) than FP32 because the larger $B_K = 32$ tiles leave room for more in-flight buffers (Table 6).

6 System-Level Optimizations

6.1 Multi-Stage Asynchronous Pipeline

The main loop is a classic circular-buffer software pipeline built on `cp.async.cg`. The warmup phase issues $\text{STAGES}-1$ asynchronous copies; the steady state commits one new copy per K -tile while consuming the oldest buffer. With six stages, up to five DMA transfers are in flight at once, hiding the 100–200 cycle L2/DRAM latency behind Tensor Core compute. A useful detail: the byte-count argument of `cp.async` is conditional setting it to zero turns a copy into a zero-fill, which handles boundary tiles without a predication branch.

The pipeline depth is chosen against the shared-memory budget: each stage holds one A and one B tile, so depth is bounded by $\lfloor \text{SMEM}_{\text{SM}} / ((B_M + B_N) B_K \text{ sizeof}) \rfloor$. The small 64×64 and 128×128 half-precision tiles fit six stages, fully hiding DRAM latency, whereas the 256×128 tile which alone consumes most of the SM’s shared memory is limited to three stages, trading some latency hiding for the higher arithmetic intensity of the larger tile.

6.2 Column-Strip CTA Grid Swizzling for L2 Reuse

CUDA launches the 2-D grid in row-major order, so consecutive CTAs share rows of C but need different columns of B , evicting each other’s B tiles from L2. We remap the

Algorithm 1 Software-pipelined main loop (per CTA)

```

1:  $ws \leftarrow 0$ 
2: for  $i \leftarrow 0$  to  $\text{STAGES} - 2$  do ▷ warmup
3:   cp.async tile  $i$  into buffer  $ws$ ; commit_group
4:    $ws \leftarrow (ws + 1) \bmod \text{STAGES}$ 
5: end for
6: cp.async.wait_group( $\text{STAGES} - 2$ ); syncthreads
7: for each  $K$ -tile  $t$  do
8:   prefetch tile  $t + \text{STAGES} - 1$  into  $ws$ ; commit
9:   load fragments via ldmatrix (double-buffered)
10:  issue all MMA calls for tile  $t$ 
11:  wait_group( $\text{STAGES} - 2$ ); syncthreads
12: end for
13: two-phase swizzled epilogue

```

linear block id into vertical column strips of height sw so that sw consecutive CTAs share the same columns of B :

```

int block_id = blockIdx.y*grid_x + blockIdx.x;
int sw = max(1, min(grid_y, CAP)); // CAP=16 FP32, 8 BF16
int bx = (block_id / sw) % grid_x;
int by = (block_id % sw)
        + (block_id / (grid_x*sw)) * sw;

```

The first sw block ids all resolve to the same bx (hence the same B column tile) and walk down distinct by ; once tile 0 pulls that B slice into L2, tiles $1 \dots sw-1$ hit it. For $grid_x=6$, $grid_y=4$, $sw=4$, ids 0–3 map to $(bx, by)=(0,0), (0,1), (0,2), (0,3)$ and id 4 to $(1,0)$ a clean column-first walk. The cap (16 for FP32, 8 for the larger-footprint BF16 tiles) is empirical: beyond it the working set exceeds L2 set associativity and reuse plateaus. On the A6000 Ada’s 96 MB L2, an 8-column strip of a typical FC1 B (768×3072 , BF16, ≈ 4.7 MB) is a ~ 2.4 MB slice that stays resident across the strip; the benefit is largest on big-batch shapes whose full B exceeds L2.

6.3 Double-Buffered Register Fetch

Within a K -tile, the kernel keeps two register fragment sets (**frA**[2], **frB**[2]) and prefetches the next K -slice into one while the MMA consumes the other, overlapping **ldmatrix** latency with MMA throughput and preventing inter-slice warp stalls.

7 Dispatch, GEMV, and Split-K

7.1 Dispatch and Tile Selection

Dispatch first routes by SM version (SM89 custom kernels; SM86 fallback; legacy CUTLASS-style variants below that). Within SM89 a shape heuristic picks the largest tile that the problem can fill: 256×128 when $M \geq 256, N \geq 128$; then 128×128 ; then 64×64 ; then a 32×32 fallback. The Split-K decision is made before tile selection.

For SM86 (Ampere) the library ships a wider set of pre-tuned variants generated by CUTLASS-style tem-

plate expansion for the NT layout alone these include $256 \times 128/16 \times 3$, $128 \times 256/16 \times 3$, $128 \times 128/16 \times 5$, $64 \times 64/32 \times 6$, and several rectangular tiles. Selection compares the total tile count ($\lceil M/128 \rceil \lceil N/128 \rceil B$) against the SM count: large problems take the 256×128 tile for arithmetic intensity, while problems that would leave SMs idle take a 64×64 tile that produces more CTAs to fill the machine. This is the same occupancy-versus-intensity tradeoff that Split-K resolves along the K axis.

7.2 GEMV Fast Path

When M or N is very small (≤ 16), a large GEMM tile wastes most of its threads on zero work. Dispatch detects this and routes to dedicated GEMV kernels (NN/NT/TN per precision), including vectorized variants for tall-and-thin and non-square-padded shapes.

7.3 Split-K for Low-Occupancy Gradient Shapes

The weight-gradient GEMM $dW = X^\top dY$ has a very large K ($B \times T$, e.g. $16 \times 1024 = 16,384$) but small M, N . With a 256×128 tile this yields only $\lceil 768/256 \rceil \times \lceil 3072/128 \rceil = 72$ CTAs against the A6000 Ada’s 142 SMs a $\sim 51\%$ occupancy floor. Split-K partitions K into P slices computed by independent CTAs, multiplying the CTA count by P and filling the machine (Table 7). We trigger it when occupancy is below $\sim 75\%$ and K is large enough to split:

Table 7: Effect of Split-K on the FC1 weight-gradient shape ($K = 16,384$, 256×128 tile, 142 SMs).

	CTAs	K per CTA	SM fill
$P = 1$	72	16,384	$\sim 51\%$
$P = 4$	288	4,096	$2\times$ over
$P = 8$	576	2,048	$4\times$ over

```

if (total_blocks*4 < sm_count*3 && K >= 512)
    splitK = min(16, ceil_div(sm_count, total_blocks));

```

Partial results are combined with **atomicAdd**. Because C must start at zero for the atomics, a small pre-scale pass applies $C *= \beta$ first and the GEMM then runs with $\beta = 1$. The cost is non-deterministic floating-point accumulation order: over a full GPT-2 run ($\sim 19,073$ steps) this shifts validation loss by $\approx 5.7 \times 10^{-3}$ relative to cuBLAS’s deterministic workspace reduction, and can be disabled by forcing $P = 1$ for bit-reproducible runs.

8 Implementation Notes

Why PTX rather than intrinsics. Several components have no CUDA C equivalent and must be written in inline PTX. The WMMA API exposes only **m16n16k8 TF32** with a different operand count, not

the `m16n8k8` shape used here; `ldmatrix` has no C intrinsic at all; and while `cp.async` is reachable through `__pipeline_memcpy_async()`, only the PTX form gives direct control over the conditional byte-count argument used for branch-free boundary handling and over explicit `commit_group/wait_group` management.

__launch_bounds__ strategy. Large tiles are launched with `__launch_bounds__(THREADS, 1)`. Setting the minimum CTAs per SM to one tells the compiler not to chase an occupancy target it cannot meet a 256×128 tile already consumes most of the SM’s shared-memory budget so it keeps the double-buffered fragment arrays `frA[2]`, `frB[2]` in registers instead of spilling them to local memory.

Shared-memory reuse in the epilogue. The buffer that held the A/B tiles during the K -loop is repurposed as the epilogue staging buffer `s_out`, avoiding a second shared-memory allocation that would further cut occupancy. This is sound only because a `__syncthreads()` after `cp.async.wait_group 0` guarantees every warp has finished reading the last tile before any warp writes epilogue data:

```
asm volatile("cp.async.wait_group 0;\n");
__syncthreads();           // last A/B tile fully consumed
__half* s_out = h_smem;    // safe to overwrite
```

9 Evaluation

Setup. All measurements are on an NVIDIA RTX 6000 Ada Generation (“A6000 Ada,” AD102, SM89, 142 SMs, 48 GB GDDR6, 96 MB L2, ~ 960 GB/s), CUDA 12.x, driver 545+. Throughput is reported as TFLOPS = $2BMNK/t$. We compare against cuBLAS and a PyTorch `matmul` baseline on the layer shapes of a GPT-2-scale model, exercising forward (NN), gradient (TN/NT), and large-batch configurations.

Timing and correctness protocol. Each configuration is run after a warmup of discarded iterations to reach steady-state clocks, then timed over many launches with the kernel bracketed by CUDA events; we report the median to reject scheduling outliers. Numerical correctness is checked against a high-precision reference: emulated-TF32 output is compared to an FP64 GEMM, and the FP16/BF16 kernels to an FP32 reference, using element-wise relative error. The operand-bias correction and FMA epilogue are what keep the emulated-FP32 path within cuBLAS’s reported tolerance rather than drifting; we treat matching accuracy as a hard constraint, so the throughput numbers below are not bought with precision.

FP32 / TF32 (Table 8, Fig. 1). Gains are modest (+2 to +4%) on large regular shapes, because cuBLAS’s own TF32 kernels are already well tuned. The advantage

Table 8: FP32 (TF32-emulated) TFLOPS on GPT-2 layer shapes.

Layer	PyT	cuBLAS	BluBLAS	Δ
QKV proj.	90.0	95.0	93.0	−2.1%
MLP FC2 proj.	92.5	97.5	99.5	+2.0%
Small feature map	23.0	23.5	23.0	−2.1%
Attention output	83.0	90.0	91.5	+1.6%
MLP FC1 expand	91.0	96.0	99.0	+3.1%
Gradient dX (FC1)	56.0	60.0	62.5	+4.1%

Table 9: FP16 TFLOPS on GPT-2 layer shapes.

Layer	PyT	cuBLAS	BluBLAS	Δ
QKV proj.	137	156	177	+13.4%
Attention output	116	148	164	+10.8%
MLP FC1	164	182	174	−4.4%
MLP FC2	154	167	189	+13.1%
FC1 gradient (TN)	155	171	214	+25.1%
FC2 gradient (TN)	158	172	217	+26.1%
Large batch	145	164	193	+17.6%
Large batch (T)	107	164	195	+18.9%

concentrates on gradient paths, where Split-K restores the occupancy cuBLAS leaves on the table; the FMA epilogue and operand-bias correction mean these gains do not cost accuracy.

FP16 (Table 9, Fig. 2). The signature result is the TN gradient shapes at +25 to +26%, peaking at 217 TFLOPS. This is almost entirely attributable to the XOR swizzle removing bank conflicts that cuBLAS’s TN kernels incur on these exact shapes.

BF16 (Table 10, Fig. 3). The largest speedup in the library is the BF16 TN gradient at +40% (226 TFLOPS), roughly +40% (226 TFLOPS); the remaining gap is the epilogue staging overhead and DRAM bandwidth on the K -reduction partials. Conversely, on the small square-ish forward shapes (QKV, attention output, 768×768), cuBLAS is 12–21% faster: its persistent kernels and finer tile granularity beat our 256×128 tile’s higher per-CTA overhead at this size. This is the principal remaining gap.

Crossover. On small square matrices cuBLAS wins on launch overhead and tile granularity; our kernels reach parity and then exceed cuBLAS once $M, N, K \gtrsim 512$ (FP32) or $\gtrsim 256$ (FP16/BF16), the point at which the deep pipeline and L2 reuse begin to pay off.

Where the gains come from. The pattern across precisions is consistent and lets us attribute the speedups to specific mechanisms rather than to a single monolithic kernel. The TN gradient shapes, which produce the largest margins (+25–26% FP16, +38–40% BF16), are exactly

Table 10: BF16 TFLOPS on GPT-2 layer shapes. Bold = win over cuBLAS.

Layer	PyT	cuBLAS	BluBLAS	Δ
QKV proj.	164	201	177	-11.9%
Attention output	141	198	156	-21.2%
MLP FC1	122	163	176	+7.9%
MLP FC2	147	180	201	+11.6%
FC1 gradient (TN)	125	160	224	+40.0%
FC2 gradient (TN)	150	163	226	+38.6%
Large batch	144	160	203	+26.8%
Large batch (T)	103	160	204	+27.5%

the shapes where (i) the contraction operand is laid out so that an un-swizzled buffer collides in shared memory so the XOR swizzle of §5 converts a multi-way conflict into a single transaction and (ii) the large K leaves the GPU under-occupied, so Split-K (§7) restores the missing CTAs. cuBLAS’s TN kernels incur a measurable conflict penalty on these shapes that the custom swizzle removes outright, which is why the half-precision gradient speedups are several times larger than the FP32 ones: at FP32 the swizzle still applies, but cuBLAS’s TF32 kernels are better tuned and the headroom is smaller. The large-batch results (+27% BF16) isolate the L2 contribution: these are the shapes whose operand B exceeds L2, so the column-strip grid swizzle of §6, which keeps a B slice resident across a strip of CTAs, has the most to recover.

Distance to peak. The remaining gap to the device’s dense BF16 Tensor Core peak is not idle Tensor Core time but memory traffic: the two-phase epilogue stages every output element through shared memory once, and the Split-K partial sums are written and re-read from DRAM, so on these gradient shapes the kernel is partly bandwidth-rather than compute-bound. This also explains why the forward NN shapes, which have higher arithmetic intensity, sit closer to compute-bound and leave cuBLAS’s persistent kernels less to beat and in the small-square case, more headroom to win on launch overhead.

10 Discussion and Limitations

The results support a clear thesis: the largest, most reliable wins come not from raw arithmetic but from *memory-system co-design* bank-conflict-free swizzling, L2-resident scheduling, and occupancy recovery via Split-K. The arithmetic tricks (TF32 bias correction, FMA epilogue) are about *not losing* accuracy while taking the Tensor Core path, rather than about speed per se.

The honest limitations are: (i) cuBLAS still wins on small square forward shapes, where our fixed large tile is mismatched and our per-CTA overhead dominates; (ii) Split-K trades a small, quantified amount of determinism for occupancy; (iii) the library is specialized to

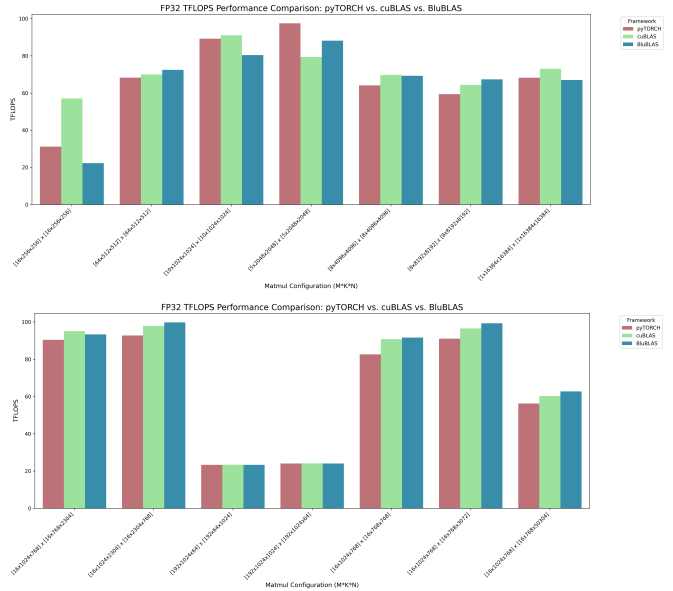


Figure 1: FP32 (TF32) TFLOPS: PyTorch vs. cuBLAS vs. BluBLAS across matmul configurations.

SM89/SM86 and to GPT-2-class shapes, and its tile heuristics would need retuning for other architectures or aspect ratios.

11 Related Work

Reduced precision on Tensor Cores. The programmability and numerical behavior of Tensor Cores have been studied since their introduction; Markidis et al. [5] characterize the precision loss of half-precision accumulation and motivate FP32 accumulation of reduced-precision products, the same choice our half-precision kernels make. Mixed-precision training [6] established FP16 with loss scaling and FP32 master weights as a practical recipe, and Kalamkar et al. [3] showed BF16’s wider exponent removes the need for loss scaling which is why our BF16 path is the preferred training format despite lower mantissa precision. Our TF32 operand-bias correction is in the same spirit: a cheap mechanism to take a reduced-precision datapath without paying its accuracy cost.

GEMM libraries and code generation. NVIDIA’s CUTLASS [8] provides the template machinery tiled MMA iterators, `cp.async` pipelines, swizzled shared-memory layouts that our SM86 fallback variants resemble, and the PTX ISA [7] documents the `mma`, `ldmatrix`, and `cp.async` instructions we issue directly. Where CUTLASS aims for a portable, exhaustively-tunable template space, we instead specialize a small set of kernels to one architecture and one workload, which is what permits hand-chosen swizzles and grid mappings rather than search-derived ones.

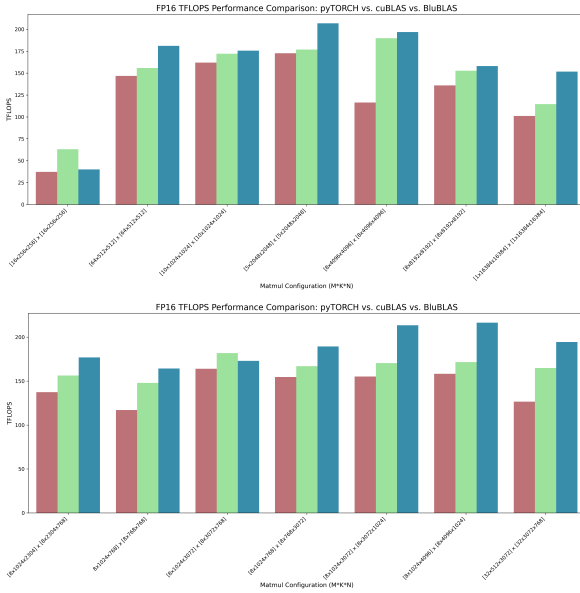


Figure 2: FP16 TFLOPS: PyTorch vs. cuBLAS vs. BluBLAS. The TN gradient shapes (right group) show the largest margins.

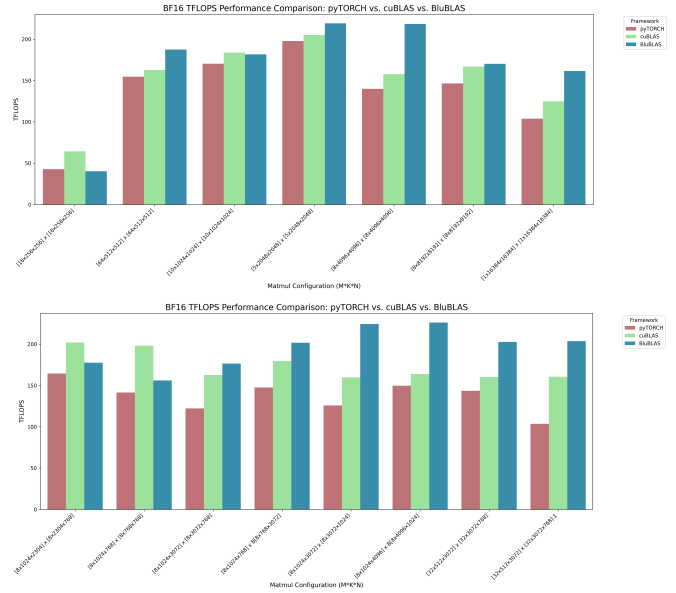


Figure 3: BF16 TFLOPS: PyTorch vs. cuBLAS vs. BluBLAS. The +40% TN-gradient result is the library’s signature; small forward shapes (left) still favor cuBLAS.

Hand-tuned GEMM worklogs. A line of detailed public worklogs establishes that hand-written kernels can iteratively close most of the gap to cuBLAS by attacking memory hierarchy bottlenecks one at a time: Boehm’s CUDA matmul worklog [1] walks from a naive kernel to near-cuBLAS through coalescing, shared-memory tiling, and register blocking, and a more recent SGEMM study [10] pushes single-precision GEMM to competitive throughput on modern NVIDIA GPUs. Our work follows the same empirical, profile-driven methodology but targets the Tensor Core datapath across three precisions and all three transformer layouts, and adds the precision-emulation and Split-K machinery those single-precision/forward-focused worklogs do not address. A concurrent preprint [11] also revisits GPU GEMM optimization on recent hardware.

Shared-memory swizzling. The XOR address-permutation technique we use throughout (§4.4, Appendix C) is analyzed in detail by Mao [4], who derives the bank-conflict-free swizzles that map logical tile coordinates to physical shared-memory banks; our per-layout masks are concrete instantiations of that principle specialized to the NN/NT/TN operand strides and the `ldmatrix` fragment layout.

Work decomposition. Split-K is long-standing folklore in high-performance GEMM for restoring occupancy on tall- K problems; its cost is the non-deterministic atomic reduction we quantify in §7. Stream-K [9] generalizes it to a work-centric decomposition that distributes K -loop iterations across SMs to balance load for arbitrary shapes, and is the natural successor to our fixed-tile Split-

K , particularly for the non-power-of-two shapes where our static heuristic is coarsest.

BluBLAS shows that, on a fixed architecture and a known workload, hand-written PTX GEMM kernels recover substantial performance over cuBLAS up to +40% in BF16 on gradient shapes without sacrificing numerical accuracy. The wins are dominated by memory-system co-design rather than arithmetic.

Future work follows directly from the limitations:

1. *Closing the small-shape gap*: asymmetric tiles (e.g. 64×128) for non-square forward shapes such as 1024×2304 , and a 2-stage pipeline for $K < 512$ to cut warmup synchronization.
2. *Stream- K decomposition* [9] to replace fixed Split- K with a work-centric split that load-balances the K -loop across SMs for arbitrary, non-power-of-two shapes.
3. *Addmm and fused epilogues*: adding an `Addmm` kernel that fuses matrix multiplication with bias addition in the epilogue, eliminating a separate memory pass for the common linear-layer pattern. Future extensions will fuse activation functions such as ReLU, GeLU, and SwiGLU directly into the GEMM epilogue so feed-forward blocks can avoid additional elementwise kernel launches and intermediate reads/writes.
4. *FP8 datapaths* (E4M3 for activations, E5M2 for gradients with loss scaling) for a theoretical $2\times$ over FP16/BF16 on Ada.
5. *Warp specialization*: dedicating one of Ada’s four warp schedulers to `cp.async/ldmatrix` production and

three to MMA consumption, targeting the memory-bound shapes identified in §9.

6. *Persistent and fused attention kernels*: a persistent CTA that processes a sequence of small attention GEMMs to amortize launch overhead, and a fused $QK^\top \rightarrow \text{softmax} \rightarrow \text{value}$ kernel that avoids materializing the $O(N^2)$ score matrix for long contexts [2].
7. *Porting to newer architectures*. The current kernels target Ada (SM89), which lacks several features that later datacenter GPUs expose. On **Hopper** (SM90), the Tensor Memory Accelerator (TMA) replaces our manual `cp.async` address generation with hardware bulk copies and descriptor-based indexing, the warpgroup-wide `wgmma` instructions operate on $64 \times$ -wide tiles directly from shared memory, and distributed shared memory across a thread-block cluster enables larger cooperative tiles together motivating a producer/consumer rewrite of our pipeline and a re-derivation of the swizzles against the 128-byte TMA layout constraints. On **Blackwell** (SM100), fifth-generation Tensor Cores add native FP4/FP6 microscaling formats and a dedicated tensor-memory tier (`tcgen05`-class MMA), which would extend the precision-emulation story of §4 below FP8 and shift the epilogue-staging tradeoffs again. The layout-agnostic XOR-swizzle and L2-aware scheduling principles carry over, but the specific masks, tile shapes, and pipeline structure would need re-tuning per architecture.

References

- [1] Simon Boehm. How to optimize a CUDA matmul kernel for cuBLAS-like performance: A worklog. Blog post, 2022. <https://siboehm.com/articles/22/CUDA-MMM>.
- [2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.14135. <https://arxiv.org/abs/2205.14135>.
- [3] Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, Jiyan Yang, Jongsoo Park, Alexander Heinecke, Evangelos Georganas, Sudarshan Srinivasan, Abhisek Kundu, Misha Smelyanskiy, Bharat Kaul, and Pradeep Dubey. A study of BFLOAT16 for deep learning training. *arXiv preprint arXiv:1905.12322*, 2019. <https://arxiv.org/abs/1905.12322>.
- [4] Lei Mao. CUDA shared memory swizzling. Blog post. <https://leimao.github.io/blog/CUDA-Shared-Memory-Swizzling/>.
- [5] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S. Vetter. NVIDIA tensor core programmability, performance & precision. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2018. arXiv:1803.04014. <https://arxiv.org/abs/1803.04014>.
- [6] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.03740. <https://arxiv.org/abs/1710.03740>.
- [7] NVIDIA. Parallel thread execution ISA. CUDA Toolkit Documentation. <https://docs.nvidia.com/cuda/parallel-thread-execution/>.
- [8] NVIDIA. CUTLASS: CUDA templates for linear algebra subroutines. GitHub repository, 2017. <https://github.com/NVIDIA/cutlass>.
- [9] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. Stream-K: Work-centric parallel decomposition for dense matrix-matrix multiplication on the GPU. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2023. arXiv:2301.03598. <https://arxiv.org/abs/2301.03598>.
- [10] Amanzhol Salykov. Advanced matrix multiplication optimization on NVIDIA GPUs. Blog post, 2025. <https://salykova.github.io/sgemm-gpu>.
- [11] Songqiao Su, Xiaofei Sun, Xiaoya Li, Albert Wang, Jiwei Li, and Chris Shum. CUDA-L2: Surpassing cuBLAS performance for matrix multiplication through reinforcement learning, 2025. arXiv:2512.02551. <https://arxiv.org/abs/2512.02551>.

A Key PTX Instruction Reference

Table 11 lists the PTX instructions issued directly by the kernels, with their tile widths and roles. None has a complete CUDA C equivalent at the level of control required (§8).

Table 11: PTX instructions used by BluBLAS.

Instruction	Width	Use
<code>mma.sync.aligned.m16n8k8.row.col.f32.tf32.tf32.f32</code>	$16 \times 8 \times 8$	FP32/TF32 accumulation
<code>mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32</code>	$16 \times 8 \times 16$	FP16 accumulation
<code>mma.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32</code>	$16 \times 8 \times 16$	BF16 accumulation
<code>ldmatrix.sync.aligned.m8n8.x4.shared.b16</code>	$4 \times 8 \times 8$ half	load 4 tiles, normal
<code>ldmatrix.sync.aligned.m8n8.x4.trans.shared.b16</code>	$4 \times 8 \times 8$ half	load 4 tiles, transposed
<code>ldmatrix.sync.aligned.m8n8.x2.shared.b16</code>	$2 \times 8 \times 8$ half	load 2 tiles, normal
<code>cp.async.cg.shared.global [dst],[src],16,bytes</code>	16 bytes	async L2→SMEM copy
<code>cp.async.commit_group</code>	—	close an async group
<code>cp.async.wait_group N</code>	—	wait until $\leq N$ groups pending

B Tile Naming Convention

Kernel variants follow the scheme `<dtype>gemm.strided.batched.<layout>.<BM>x<BN>.<BK>x<stages>`. For example:

- `Sgemm.nt.256x128.16x3` — FP32, NT layout, 256×128 tile, $B_K = 16$, 3 pipeline stages.
- `Hgemm.tn.128x128.32x5` — FP16, TN layout, 128×128 tile, $B_K = 32$, 5 stages.
- `Bgemm.nn.64x64.32x6` — BF16, NN layout, 64×64 tile, $B_K = 32$, 6 stages.

C Shared-Memory Layout Design

We considered three families of shared-memory layout before settling on XOR swizzling. A *naive* row-major layout (`As[r*cols+c]`) mirrors HBM and is conflict-free for row access, but the MMA’s column-strip access pattern collides 8 ways. A *padded* layout (`As[r*(cols+pad)+c]`) breaks the collision by shifting each row, but wastes $B_M \cdot \text{PAD}$ elements of shared memory per stage which directly costs pipeline depth and occupancy and produces misaligned addresses that defeat the vectorized 16-byte instructions. Load-time *transposition* removes the column-strip pattern but needs an extra buffer or complex indexing and can introduce conflicts during the transpose itself. XOR swizzling dominates all three on the axes that matter here (Table 12), which is why every operand and the epilogue use it.

Table 12: Why XOR swizzling over the alternatives.

Criterion	XOR swizzle	Padding	Transpose
Extra shared memory	none	yes	sometimes
Extra instructions	1 XOR	1 ADD	many
Conflict-free	yes	yes	maybe
Invertible	trivially	n/a	n/a
Layout-agnostic	yes	yes	no

Tables 13 and 14 give the complete per-layout store/read formulas as implemented. The half-precision periods are half the FP32 periods for the same B_K because a 16-byte vector holds 8 half-precision elements but only 4 floats, so the same row width occupies half as many banks.

Table 13: FP32 (SM86) complete swizzle map.

Layout	Matrix	Store	Read
NT/NN	A	$c^{\wedge}(((r\&7)<<2)\&(BK-1))$	$c^{\wedge}maskA[i]$ (precomp.)
TN	A	$c^{\wedge}((r\&7)<<3)$	$m^{\wedge}((k\&7)<<3)$ (inline)
NT	B	$c^{\wedge}(((r\&7)<<2)\&(BK-1))$	$c^{\wedge}mask$ (precomp.)
TN/NN	B	$c^{\wedge}(((r\&7)<<2)\&(BN-1))$	$n^{\wedge}((k\&7)<<2)$ (inline)
grid	—	column-strip, $sw \leq 16$	—

Table 14: BF16/FP16 complete swizzle map. Epilogue swizzle is unique to the half-precision kernels.

Context	Row bits	Period	Formula
TN-A	K-row	$\&7$	$lc^{\wedge}((lr\&7)<<3)$
NT/NN-A	M-row	$\&3$	$c^{\wedge}((r\&3)<<3)$
NT-B	N-row	$\&3$	$lr^{\wedge}((lc\&3)<<3)$
TN/NN-B	K-row	$\&7$	$c^{\wedge}((r\&7)<<3)$
Epilogue	$c/8$	$\&7$	$r^{\wedge}((c/8)\&7)$, second row $\wedge 8$
grid	—	—	column-strip, $sw \leq 8$